

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer represents the Arrival Time (AT) and the second integer represents the Burst Time (BT).

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
3
0 2
1 2
2 3
```

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 1.00

Example 2

Sample Input 2

3

0 5

5 3

8 2

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33

Average Waiting Time: 0.00

Solution:

```
#include <stdio.h>
```

```
int main() {
```

```

int n, i;

int at[100], bt[100], ct[100], tat[100], wt[100];

float avg_tat = 0, avg_wt = 0;


printf("Enter number of processes: \n");

scanf("%d", &n);


for(i = 0; i < n; i++) {

    printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);

    scanf("%d %d", &at[i], &bt[i]);

}


for(i = 0; i < n; i++) {

    if(i == 0) {

        ct[i] = at[i] + bt[i];

    }

    else {

        if(ct[i-1] < at[i])

            ct[i] = at[i] + bt[i];

        else

            ct[i] = ct[i-1] + bt[i];

    }


    tat[i] = ct[i] - at[i];

    wt[i] = tat[i] - bt[i];


    avg_tat += tat[i];

    avg_wt += wt[i];

```

```
}
```

```
avg_tat /= n;
```

```
avg_wt /= n;
```

```
printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
```

```
printf("Average Waiting Time: %.2f\n", avg_wt);
```

```
return 0;
```

```
}
```

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $\text{Turnaround Time (TAT)} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$
- $\text{Waiting Time (WT)} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
4
0 6
0 8
0 7
0 3
```

Sample Output 1

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00

Average Waiting Time = 7.00

Example 2

Sample Input 2

3

0 3

5 2

5 1

Sample Output 2

Enter number of processes:

Enter Arrival Time and Burst Time for P1:

Enter Arrival Time and Burst Time for P2:

Enter Arrival Time and Burst Time for P3:

Average Turnaround Time = 2.33

Average Waiting Time = 0.33

Solution:

```
#include <stdio.h>
```

```

int main() {

    int n, i, j, time = 0, min, idx;

    int at[100], bt[100], ct[100], tat[100], wt[100], done[100];

    float avg_tat = 0, avg_wt = 0;


    printf("Enter number of processes: \n");

    scanf("%d", &n);


    for(i = 0; i < n; i++) {

        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);

        scanf("%d %d", &at[i], &bt[i]);

        done[i] = 0;

    }


    int completed = 0;


    while(completed < n) {

        min = 9999;

        idx = -1;


        for(i = 0; i < n; i++) {

            if(at[i] <= time && done[i] == 0) {

                if(bt[i] < min) {

                    min = bt[i];

                    idx = i;

                }

            }

        }

    }

```

```
}

if(idx == -1) {
    time++;
} else {
    time += bt[idx];
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];

    avg_tat += tat[idx];
    avg_wt += wt[idx];

    done[idx] = 1;
    completed++;
}
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
printf("Average Waiting Time = %.2f\n", avg_wt);

return 0;
}
```